WERGONIC · QUALITY TEAM

QA Onboarding & Study Guide

Welcome aboard. This guide explains what your job is, what to study first, and exactly how you'll work with us — step by step, starting from the basics.

For: our new QA team member
Level: entry — no prior testing experience needed
Read this first, then keep it as a reference

1 · Welcome — what your job really is

Don't worry about knowing everything on day one. This is a learn-as-you-go role, and this guide is your map.

Your mission in one sentence: **use our products, find the things that are broken or confusing, and report them clearly so the developers can fix them** — while gradually helping us build a library of automatic tests so those bugs never come back.

You are the person who cares about quality more than anyone else. Developers build features fast and sometimes miss things — that's normal. Your job is to be the safety net: the calm, careful person who tries every button, every form, every weird situation, and catches problems before our customers do.

> **Two things make a great QA — and neither needs a computer-science degree:**
>
> - **Curiosity** — always asking "what happens if I do this instead?"
> - **Care** — writing down problems clearly, with enough detail that someone else can understand and fix them.

You'll do **two kinds of work**, and you'll grow into the second one over time:

- **Manual testing (start here, day one):** using the app by hand, feature by feature, and writing a bug ticket for anything wrong. You can do this immediately, no coding needed.
- **Automatic tests (grow into this):** writing small pieces of code that check the app for us automatically. You won't do this alone — you'll use **Claude** (our AI coding assistant) to help you write them, and you'll learn as you go. Some tests are already set up as examples for you to follow.

2 · The words you'll hear (keep this handy)

You'll hear these every day. You don't need to memorize them — just refer back here whenever one confuses you.

Word	Plain meaning
Bug	Something in the app that doesn't work the way it should.
Feature	A thing the app can do (e.g. "log in", "start a session", "download a report").
Test case	A written step-by-step: "do these steps, expect this result." Your main manual-testing tool.
Manual test	You, a human, using the app and checking it by hand.
Automatic test	Code that checks the app for you, in seconds, over and over.
Small logic test	An automatic test for one small calculation. (Developers call this a "unit test".)
Full-journey test	An automatic test that clicks through the app like a real user. (Called an "end-to-end" or "E2E" test.)
The automatic checker	The system that runs all our tests automatically whenever code changes. (Real name: "CI/CD" or "the pipeline".)
Regression	When something that used to work breaks again. Our tests exist mainly to catch these.
Staging	A private copy of the app where we test safely before customers see it.
Production	The real, live app that customers use. Never test experiments here.
Git	The tool developers use to store and share code. You'll learn just enough to get the code and run the app.
Ticket	A task or bug written down in our project tool, so the team can track and fix it.

3 · What to study — your learning path

Go in order. Don't jump ahead to code before you're comfortable with the basics. Spend real time on each phase.

1 Testing fundamentals START HERE · ~1–2 weeks

Learn what testing is, the types of tests, how to design a test case, and how to report a bug well. This is the single most important phase — it's the core of your job and needs no coding.

- **ISTQB Foundation** — the standard entry-level QA syllabus. Read it and consider the certificate later. (istqb.org)
- **Test Automation University** — free courses, start with the manual/fundamentals track. (testautomationu.applitools.com)
- **Ministry of Testing** — friendly community and beginner articles. (ministryoftesting.com)

2 Learn our products by using them ~2–4 weeks, overlaps with the rest

Get accounts for our web apps and the mobile app. Use every feature like a curious customer. Start filing bug tickets immediately — this is real, valuable work from week one. Ask the team for a feature list so you know what to cover.

3 How the web works + a little Git ~2 weeks

Just enough to understand what you're testing and to run the apps yourself.

- **How websites work** (client, server, and the "API" that connects them) — MDN "How the web works". (developer.mozilla.org)
- **Git basics** — how to get the code and switch versions. Try "Learn Git Branching". (learngitbranching.js.org)

4 Writing automatic tests (with Claude's help) ~from week 5, ongoing

Now you start adding automatic tests. You are *not* expected to write these from scratch alone — you'll read the examples already set up, then use **Claude** to help you write more. Learn the basics so you can read, run, and fix what Claude produces.

- **A little JavaScript/TypeScript** (our web apps) — freeCodeCamp basics. (freecodecamp.org)
- **Playwright** — the tool for full-journey web tests; its "Getting Started" is beginner-friendly. (playwright.dev)
- **A little Python + pytest** (our backend) — come to this last, once the web side feels comfortable.

> **Pace yourself.** Phases 1 and 2 make you useful in your first weeks. Phases 3 and 4 turn you into a strong QA over the following months. It's completely fine if the coding parts take time — everyone starts here.

4 · Your day-to-day responsibilities

This is the loop you'll repeat, feature by feature, until every part of every product is covered.

Responsibility	What it looks like in practice
Test features by hand	Go through the app methodically (see Section 6). Try the normal path, then try to break it.
File a bug ticket for every problem	One ticket per bug, clear and detailed (see Section 5). This is your most important output.
Write automatic tests	With Claude's help, turn the important checks into automatic tests so they run forever.
Re-test after fixes	When a developer says a bug is fixed, confirm it yourself before closing the ticket.
Track coverage	Keep a simple list of every feature and whether you've tested it, so nothing is forgotten.
Watch for regressions	After each release, quickly re-check the main flows to make sure nothing old broke.

The core loop

Pick a feature → test it every way you can → find a bug → write a clear ticket → developer fixes it → you confirm the fix → move to the next feature.

Repeat until you've covered everything. Then start again on the next release. Quality is never "done" — it's a habit.

> **Good news: a lot is already set up for you.** The automatic checker (the pipeline) and a few example tests are ready. You're not building from an empty page — you're following patterns that already exist and expanding them.

5 · How to write a great bug ticket

A bug ticket is only useful if someone else can understand and reproduce it. This is a skill — and it's the one that makes people respect a QA.

The golden rule: **a developer should be able to read your ticket and reproduce the bug without asking you a single question.** Always include what you did, what happened, and what you expected instead.

Title	Short and specific. "Report page crashes when session has no data" — not "report broken".
Product	Which app: client panel / admin / mobile / backend.
Steps to reproduce	Numbered steps, exactly what you did: 1) Log in as X. 2) Open Sessions. 3) Click an empty session.
What happened	The actual (wrong) result: "The page went blank and showed an error."
What you expected	"The page should show an empty report with a 'no data' message."
Evidence	A screenshot or short screen recording. Always attach one if you can.
Environment	Where you saw it: browser + version, or phone model, and staging vs production.
Severity	How bad it is — see the table below.

Severity — how urgent is it?

Level	Meaning
Critical	App is unusable or data is lost. Customers are blocked. Report immediately.
High	A key feature is broken, but there's a workaround.
Medium	Something works wrong but doesn't block the user.
Low	Small issue — a typo, a slightly-off color, a cosmetic glitch.

> **Before you file it:** try to reproduce the bug a second time. If it happens again, you've got a solid ticket. If it doesn't, note "happened once, couldn't reproduce" — that's still useful, just say so honestly.

6 · How to actually test a feature

Testing isn't random clicking. Here's a simple, repeatable method for any feature you're handed.

1. **The normal path first ("happy path").** Use the feature the way it's meant to be used. Does it work? Example: fill a form correctly and submit — does it save?
2. **The edges.** Try the boundaries: empty fields, very long text, the number 0, huge numbers, special characters, dates in the past/future.
3. **The wrong input.** Deliberately do it "wrong": letters where numbers go, a bad email, skipping required fields. A good app should show a clear, friendly error — not crash.
4. **Interruptions and weird situations.** What if you lose internet halfway? Refresh mid-action? Click the button twice fast? Press "back"?
5. **Different conditions.** Try another browser, a phone screen size, a different user role (admin vs normal user).
6. **Check it looks right.** Is text readable, aligned, translated? Nothing overlapping or cut off?

> **The QA mindset.** Developers test that a feature *works*. Your extra value is testing that it *doesn't break* when someone uses it unexpectedly. Always ask: "what would a confused or careless user do here?"

Using Claude to write your automatic tests

When you've manually confirmed how a feature *should* behave, you can turn that into an automatic test with Claude. A good way to ask:

- "Here's the feature and the steps a user takes. Write a Playwright test that logs in, does these steps, and checks this result."
- Then **run the test yourself** and confirm it passes. If it fails, paste the error back to Claude and ask it to fix it.
- Start by copying an existing example test and changing it — that's the fastest way to learn the pattern.

Over time you'll read these tests comfortably and know when Claude got it right. That's exactly the growth we're expecting from you.

> **Your first month, simply:** learn the basics (Section 3), use every feature by hand, and file clear bug tickets. Do that well and you're already doing the most important part of the job. The automatic tests will come naturally after.